EXPERIMENT 2

Name-Spandan Deb
Class-D20A
Roll No-13

Aim: Create a Blockchain using Python.

Theory:

1)What is Blockchain?
Blockchain is a distributed and decentralized digital ledger used to record transactions across multiple computers in a network. Instead of storing data in a single central database, blockchain stores information in the form of blocks that are linked together in chronological order.
Each block contains transaction data, a cryptographic hash of the previous block, and a timestamp, making the data secure, transparent, and tamper-resistant. Once data is recorded on the blockchain, it cannot be altered without changing all subsequent blocks, which requires network consensus.

2)What is a Block?
A block is the basic unit of a blockchain. It stores a set of verified transactions and links to the previous block using a cryptographic hash.
Blocks are connected sequentially, forming a chain known as the blockchain. The first block in the chain is called the Genesis Block.

3)Components of a Block

Each block in a blockchain typically consists of the following components:

1. **Block Header**
   Contains metadata about the block.It helps in identifying, linking, and validating the block in the blockchain.

- Previous Block Hash-It stores the hash of the previous block, which links blocks together and ensures that the blockchain cannot be altered.

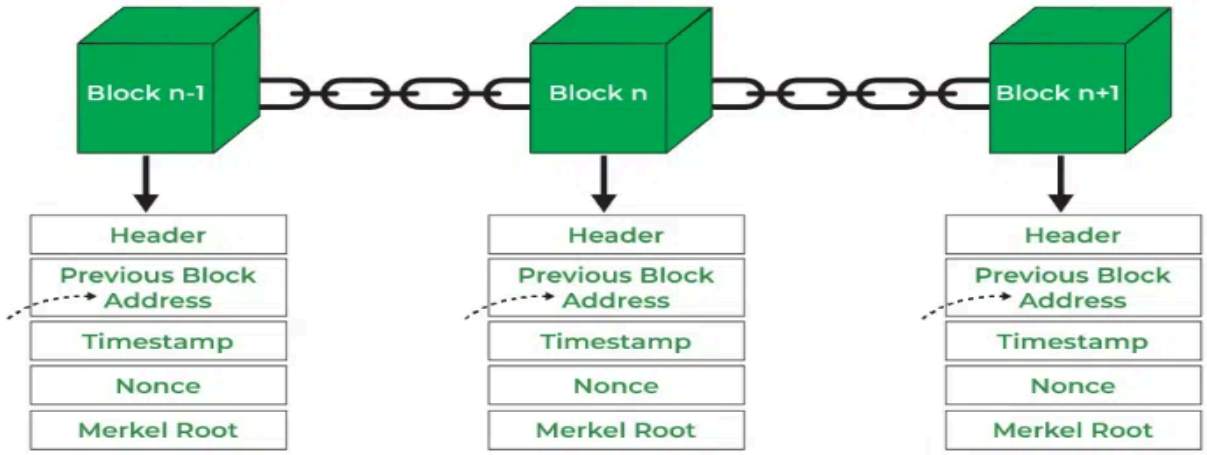
- Timestamp-The timestamp records the exact date and time when the block was created. It helps maintain the chronological order of blocks in the blockchain.

- Nonce-A nonce is a random number used during the mining process. Miners change the nonce value to generate a valid hash that meets the network's difficulty level.

- Merkle Root-The Merkle Root is a single hash value that represents all transactions in the block. It ensures efficient and secure verification of transaction data.

2. **Transaction Data**
   Stores all the transactions included in the block.

3. **Hash**
   A unique cryptographic value generated using a hashing algorithm (such as SHA-256) that identifies the block and ensures data integrity.



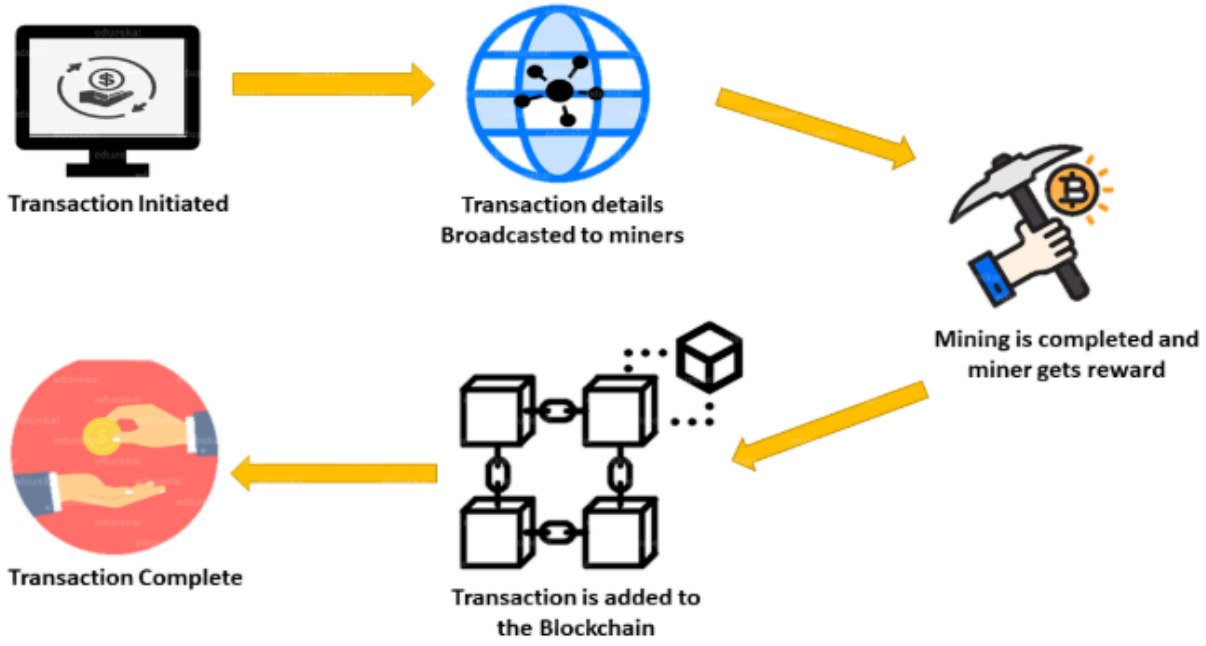
4)Process of Mining

Mining is the process of adding new blocks to the blockchain and validating transactions. It involves solving complex mathematical problems using computational power.

Steps involved in mining:

1. Transactions are collected and verified by miners.
2. Verified transactions are grouped into a block.
3. Miners compete to solve a cryptographic puzzle by finding a valid nonce.
4. The first miner to solve the puzzle broadcasts the block to the network.
5. Other nodes verify the block.
6. Once approved, the block is added to the blockchain, and the miner receives a reward.

Mining ensures security, decentralization, and consensus in the blockchain network.



5)Checking the validity of the blocks in blockchain

The validity of a block is checked using the following methods:

1. Hash Verification
   The block's hash must be correctly calculated and meet the network's difficulty requirements.

2. Previous Hash Matching
   The previous hash stored in the block must match the hash of the preceding block.

3. Transaction Validation
   All transactions in the block must be valid and digitally signed.

4. Proof of Work Validation
   The nonce must satisfy the cryptographic puzzle defined by the consensus mechanism.

5. Consensus Agreement
   The majority of nodes in the network must agree on the block's validity before it is added to the chain.

Implementation:

Installing Flask

```
PS C:\Users\Student\Downloads\BCEx2> pip install flask
Collecting flask
  Downloading flask-3.1.2-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
```

blockchain.py

```
import datetime
import hashlib
import json

class Blockchain:

    def __init__(self):
        self.chain = []
        self.difficulty = 4  # 4 leading zeroes
        self.create_genesis_block()

    def create_genesis_block(self):
        genesis_block = {
            'index': 1,
            'timestamp': str(datetime.datetime.now()),
            'data': 'Genesis Block',
            'nonce': 0,
            'previous_hash': '0'
        }
        genesis_block['hash'] = self.calculate_hash(genesis_block)
        self.chain.append(genesis_block)

    def calculate_hash(self, block):
        block_copy = block.copy()
        block_copy.pop('hash', None)
        encoded = json.dumps(block_copy, sort_keys=True).encode()
        return hashlib.sha256(encoded).hexdigest()

    def mine_block(self, data):
        previous_block = self.chain[-1]
        nonce = 0

        while True:
```

```
        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'data': data,
            'nonce': nonce,
            'previous_hash': previous_block['hash']
        }

        block_hash = self.calculate_hash(block)

        if block_hash.startswith('0' * self.difficulty):
            block['hash'] = block_hash
            self.chain.append(block)
            return block

        nonce += 1

    def is_chain_valid(self):
        for i in range(1, len(self.chain)):
            current = self.chain[i]
            previous = self.chain[i - 1]

            if current['previous_hash'] != previous['hash']:
                return False

            recalculated_hash = self.calculate_hash(current)
            if recalculated_hash != current['hash']:
                return False

            if not current['hash'].startswith('0' * self.difficulty):
                return False

        return True
```

app.py

```
from flask import Flask, jsonify
from blockchain import Blockchain

app = Flask(__name__)
blockchain = Blockchain()

@app.route('/mine_block', methods=['GET'])
def mine_block():
    block = blockchain.mine_block("Some transaction data")
    return jsonify({
        'message': '⛏️ Block mined successfully!',
        'block': block
    }), 200

@app.route('/get_chain', methods=['GET'])
def get_chain():
    return jsonify({
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }), 200

@app.route('/is_valid', methods=['GET'])
def is_valid():
    return jsonify({
        'is_valid': blockchain.is_chain_valid()
    }), 200

if __name__ == '__main__':
    print(" Mining blockchain with 4 leading zeroes...")
    app.run(debug=True, use_reloader=False)
```

127.0.0.1:5000/spandan/mine_block

Pretty-print ☑


```
{
  "block": {
    "data": "Some transaction data",
    "hash": "000061acddca8ecc38e25c84138a485908bd42091077ee2d3fc91df2ad3d61d7",
    "index": 2,
    "nonce": 156622,
    "previous_hash": "672acaa07fe2b023b154e0a37b58c904918b8225b348048d7b8c4a47f3ce1300",
    "timestamp": "2026-01-30 09:59:38.032525"
  },
  "message": "⛏️ Block mined successfully!"
}
```


127.0.0.1:5000/spandan/get_chain

Pretty-print ☐


```
{
  "chain": [
    {
      "data": "Genesis Block",
      "hash": "672acaa07fe2b023b154e0a37b58c904918b8225b348048d7b8c4a47f3ce1300",
      "index": 1,
      "nonce": 0,
      "previous_hash": "0",
      "timestamp": "2026-01-30 09:59:23.204323"
    },
    {
      "data": "Some transaction data",
      "hash": "000061acddca8ecc38e25c84138a485908bd42091077ee2d3fc91df2ad3d61d7",
      "index": 2,
      "nonce": 156622,
      "previous_hash": "672acaa07fe2b023b154e0a37b58c904918b8225b348048d7b8c4a47f3ce1300",
      "timestamp": "2026-01-30 09:59:38.032525"
    },
    {
      "data": "Some transaction data",
      "hash": "0000e0b5e9a6cec1ffdd01c5120d3c661fad712a1163e86322b9f75b731c0978",
      "index": 3,
      "nonce": 26184,
      "previous_hash": "000061acddca8ecc38e25c84138a485908bd42091077ee2d3fc91df2ad3d61d7",
      "timestamp": "2026-01-30 10:07:15.936494"
    }
  ],
  "length": 3
}
```


127.0.0.1:5000/spandan/is_valid

Pretty-print ☐


```
{
  "is_valid": true
}
```